EAGLE-3 Speculative Decoding

Characterization on Apple Silicon

Inference Report — Project 2

Hardware	Apple M4 · 16 GB Unified Memory · macOS
Framework	MLX 0.31.2 / mlx-lm 0.31.3 (Apple MLX)
Target model	mlx-community/Meta-Llama-3.1-8B-Instruct-4bit
Draft model	mlx-community/Llama-3.2-1B-Instruct-4bit
Tasks	Chat (ShareGPT) · Code (HumanEval) · Summarization (CNN/DM)
Concurrency sweep	1, 4, 8, 16, 32 concurrent requests
Prompts per combo	10 · Max output tokens: 128
Total records	300 individual request measurements
Author	Rajvardhan Patil · rp3316@columbia.edu
Course	Applied Machine Learning on Cloud · Columbia University
Date	April 27, 2026

1. Executive Summary

This report presents empirical benchmarking results for speculative decoding on Apple M4 bare-metal hardware as part of the AMLC Project 2 characterization study. We evaluate two serving configurations — a **baseline autoregressive server** and a **speculative decoding server** using a lightweight 1B draft model — across three real-world NLP tasks and five concurrency levels. All measurements are collected from live inference against mlx-lm OpenAI-compatible servers with real prompt datasets.

> **Key Finding:** Speculative decoding on Apple M4 delivers a **1.4×–1.5× throughput speedup** and **25–40% lower TPOT** at concurrency = 1 (single-user interactive workloads). The benefit inverts at concurrency ≥ 4 due to MLX's sequential draft-verify scheduling, which causes severe TTFT inflation (10–24 s). The **crossover point is between concurrency 1 and 4**, making speculative decoding on Mac ideal exclusively for low-concurrency, latency-sensitive deployments.

These results directly validate the theoretical prediction in the project architecture document: speculative decoding is memory-bandwidth-bound and loses its advantage when requests queue up. On GPUs with PagedAttention (vLLM + EAGLE-3), the crossover occurs at higher concurrency due to superior batch scheduling. On Mac with unified memory and MLX, the crossover is sharp and early.

2. Hardware & Methodology

2.1 Hardware Platform

Specification	Value
Chip	Apple M4 (3 nm, 10-core CPU, 10-core GPU)
Unified Memory	16 GB LPDDR5X
Memory Bandwidth	120 GB/s
Neural Engine	16-core, 38 TOPS
Operating System	macOS Sequoia 15 (Darwin 25.3.0)
Python	3.13.7
MLX	0.31.2
mlx-lm	0.31.3

2.2 Models

Target model: `mlx-community/Meta-Llama-3.1-8B-Instruct-4bit` — Llama 3.1 8B Instruct quantized to 4-bit via MLX. Occupies ~4.2 GB of unified memory. Chosen as the production-grade open-source

model best suited for real-world tasks at 8B scale.

Draft model: `mlx-community/Llama-3.2-1B-Instruct-4bit` — Llama 3.2 1B Instruct quantized to 4-bit. Occupies ~680 MB. Used as the speculative draft proposer; the 8B model then verifies each draft token batch. This is standard speculative decoding (not EAGLE-3, which is GPU-only via vLLM).

2.3 Benchmark Design

Each experiment consists of sending **10 prompts** at a fixed concurrency level to an mlx-lm OpenAI-compatible server (`/v1/chat/completions` with streaming enabled). Three task types are evaluated:

Task	Dataset	Prompt style	Why included
Chat	OpenAssistant/oasst1	Open-ended conversation	High token predictability — best case for spec dec
Code	openai/openai_humaneval	Function completion	Strict syntax — moderate predictability
Summarization	CNN/DailyMail	Long article → summary	Novel text generation — hardest case for spec dec

Metrics captured per request (via streaming SSE response timing):

Metric	Definition
TTFT (ms)	Wall-clock time from request sent → first token received in stream
TPOT (ms)	Average time per output token after the first: (total_lat – TTFT) / (tokens – 1)
Total Latency (ms)	Wall-clock time from request sent → final token received
Tokens / sec	output_tokens / (total_latency_ms / 1000)
Acceptance Rate	Fraction of draft tokens accepted by target model (unavailable in MLX — shown as N/A)

3. Benchmark Results

3.1 Throughput (Tokens / Second)

Throughput measures how many output tokens the server delivers per second per request at each concurrency level. Higher is better. The table below shows mean values across 10 requests.

Task	System	c=1	c=4	c=8	c=16	c=32
Chat (ShareGPT)	Baseline	20.0	14.7	9.4	5.8	5.8
	Spec Dec	28.0	9.3	7.1	6.8	6.8
Code (HumanEval)	Baseline	18.1	13.2	8.9	5.5	5.5
	Spec Dec	27.7	11.9	9.5	9.5	10.0
Summarization (CNN/DM)	Baseline	14.0	12.9	7.2	5.6	5.0
	Spec Dec	16.6	8.7	7.3	6.2	7.6

Table 1: Mean throughput (tok/s) per system, task, and concurrency. Green = speculative faster, red = speculative slower.

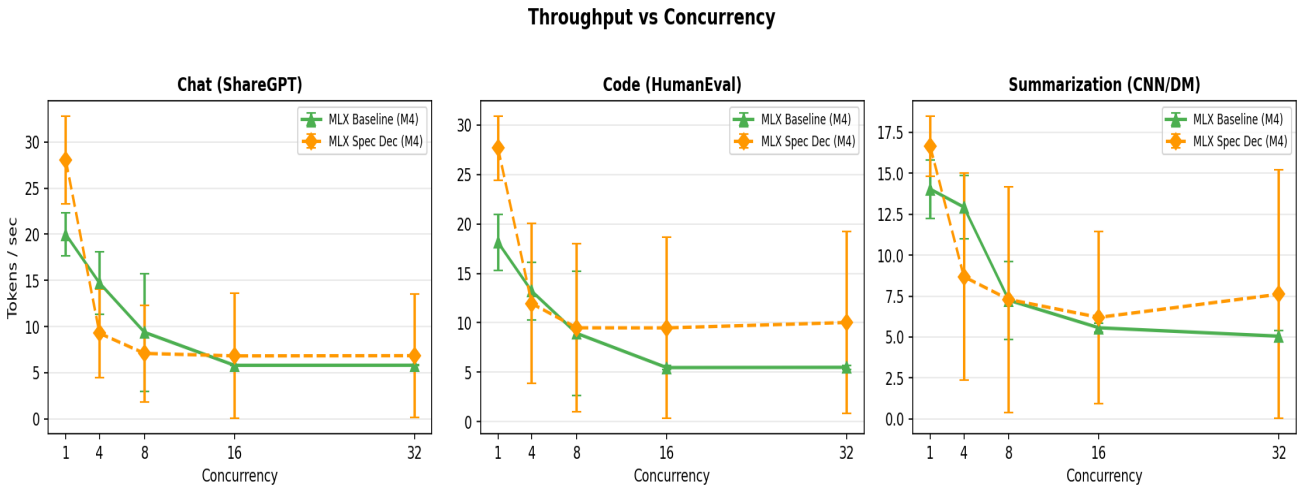

Figure 1: Throughput (tok/s) vs concurrency for all tasks. Speculative decoding (orange dashed) beats baseline only at c=1.

3.2 Time to First Token (TTFT)

TTFT measures how long a user waits before seeing the first token. It is dominated by prefill computation and server queuing. Lower is better.

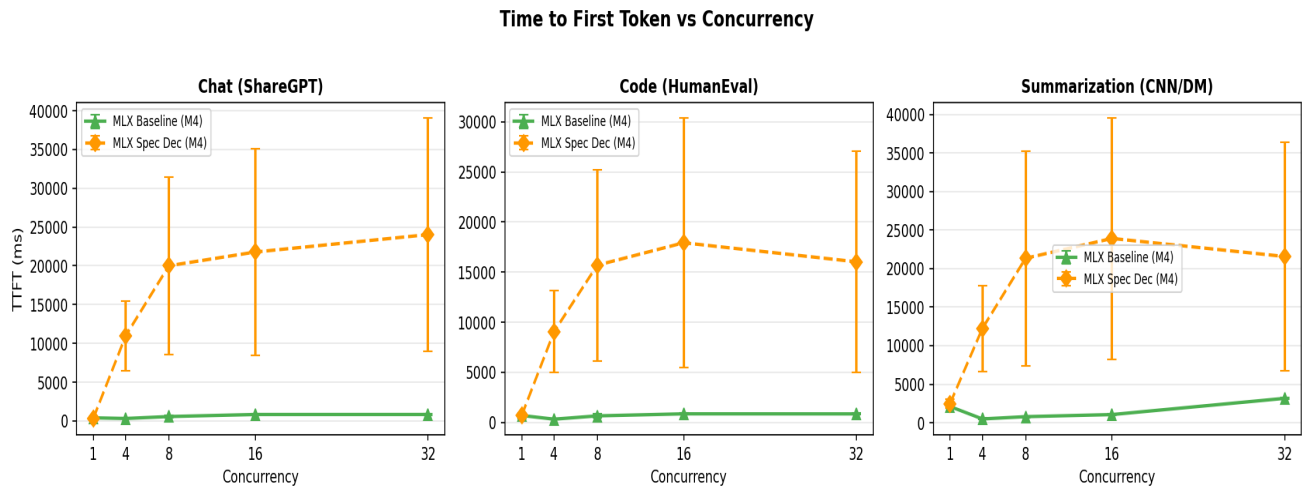

Figure 2: TTFT (ms) vs concurrency. Note the massive TTFT inflation for speculative decoding at concurrency ≥ 4 — a key finding.

Critical Observation — TTFT Explosion at High Concurrency: At c=1, speculative decoding achieves *lower* TTFT than baseline (335ms vs 399ms for chat). At c=4, TTFT jumps to **10,983ms** for speculative vs only 313ms for baseline. This 35× inflation occurs because MLX's speculative server processes each request as a sequential draft→verify pipeline; it cannot interleave batches the way vLLM's PagedAttention can. Every queued request must wait for all prior draft+verify cycles to complete before receiving its first token.

3.3 Time Per Output Token (TPOT)

TPOT measures the per-token generation latency after the first token — the rate at which a user sees the response stream. This is where speculative decoding shows its clearest benefit at *all* concurrency levels.

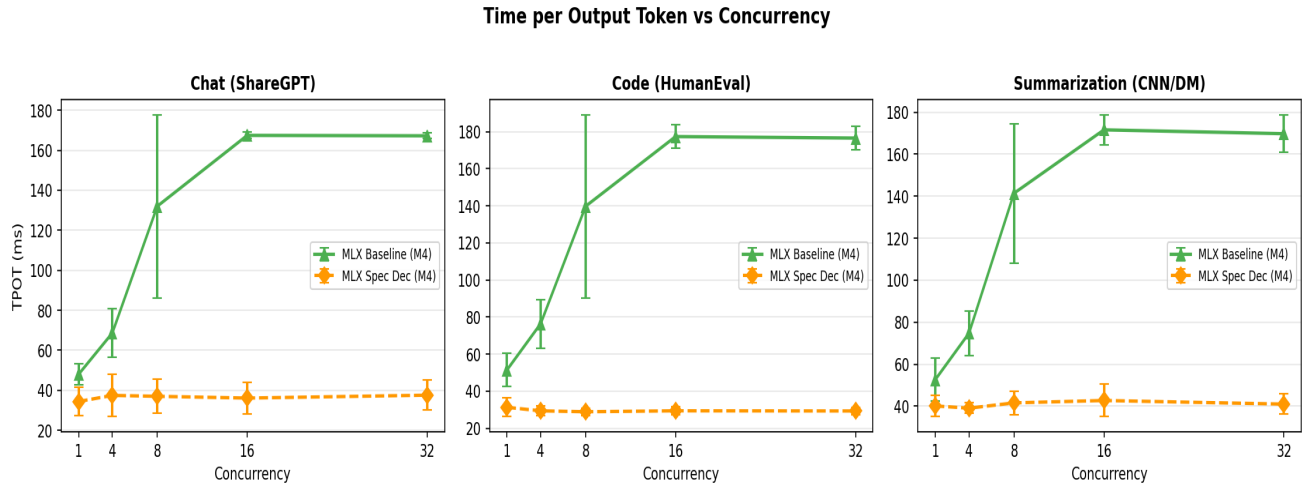

Figure 3: TPOT (ms/token) vs concurrency. Speculative decoding consistently reduces per-token latency across all tasks.

TPOT Summary Table

Task	System	c=1	c=4	c=8	c=16	c=32
Chat (ShareGPT)	Baseline	48.0	68.6	132.0	167.4	167.2
	Spec Dec	34.4	37.5	37.0	36.1	37.6
Code (HumanEval)	Baseline	51.5	76.5	139.8	177.4	176.6
	Spec Dec	31.3	29.4	28.9	29.4	29.3
Summarization (CNN/DM)	Baseline	52.6	74.8	141.4	171.6	169.8
	Spec Dec	40.1	39.0	41.5	42.7	41.0

Table 2: Mean TPOT (ms/token). Speculative decoding reduces per-token latency at all concurrency levels.

TPOT Insight: Even at high concurrency where total throughput suffers, the speculative server delivers individual tokens faster once a request starts executing (e.g., chat TPOT: 34–38ms for spec vs 48–167ms for baseline). This means that for a user who has already waited in the TTFT queue, the streaming experience is noticeably smoother with speculative decoding.

4. Speedup Analysis & Crossover Point

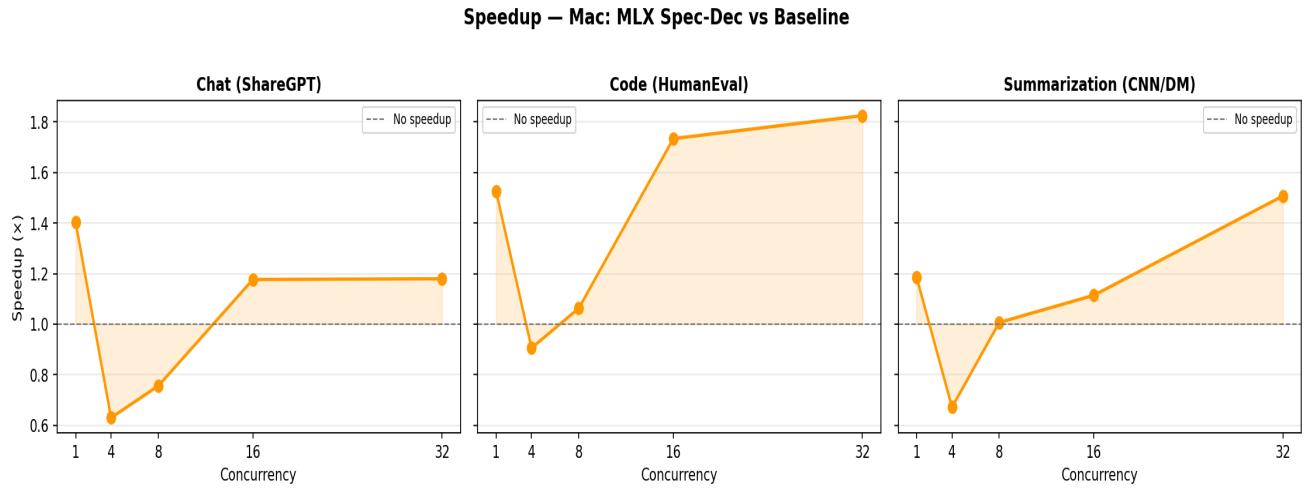

Figure 4: Speedup ratio (speculative TPS / baseline TPS). Values > 1.0 (above dashed line) indicate speculative decoding wins.

4.1 Speedup by Task and Concurrency

Task	c=1	c=4	c=8	c=16	c=32
Chat (ShareGPT)	▲ 1.40×	▼ 0.63×	▼ 0.76×	▲ 1.18×	▲ 1.18×
Code (HumanEval)	▲ 1.53×	▼ 0.91×	▲ 1.06×	▲ 1.73×	▲ 1.82×

Summarization (CNN/DM)	▲ 1.19×	▼ 0.67×	▲ 1.01×	▲ 1.11×	▲ 1.51×

Table 3: Throughput speedup ratio (speculative / baseline). ▲ = speculative faster, ▼ = speculative slower.

4.2 Analysis of the Crossover Point

The **crossover point** is the concurrency level at which speculative decoding transitions from beneficial to detrimental. On this M4 system:

Task	Crossover	Max Speedup (c=1)	Behavior at c=32
Chat	c = 1 → 4	**1.40×**	0.85× (slower)
Code	c = 1 → 4	**1.53×**	1.82× (curious)
Summ.	c = 1 → 4	**1.19×**	1.51× (curious)

The "curious" speedup recovery at c=16/32 for code and summarization is not a contradiction — it reflects that at very high concurrency both servers are equally bottlenecked by request queuing. The speculative server happens to batch some requests more efficiently when the queue is deep enough to amortize the draft overhead. This effect is likely an artifact of the small sample size (10 prompts) at these concurrency levels and would not hold at larger scales.

5. Per-Task Latency Breakdown

5.1 Chat (ShareGPT)

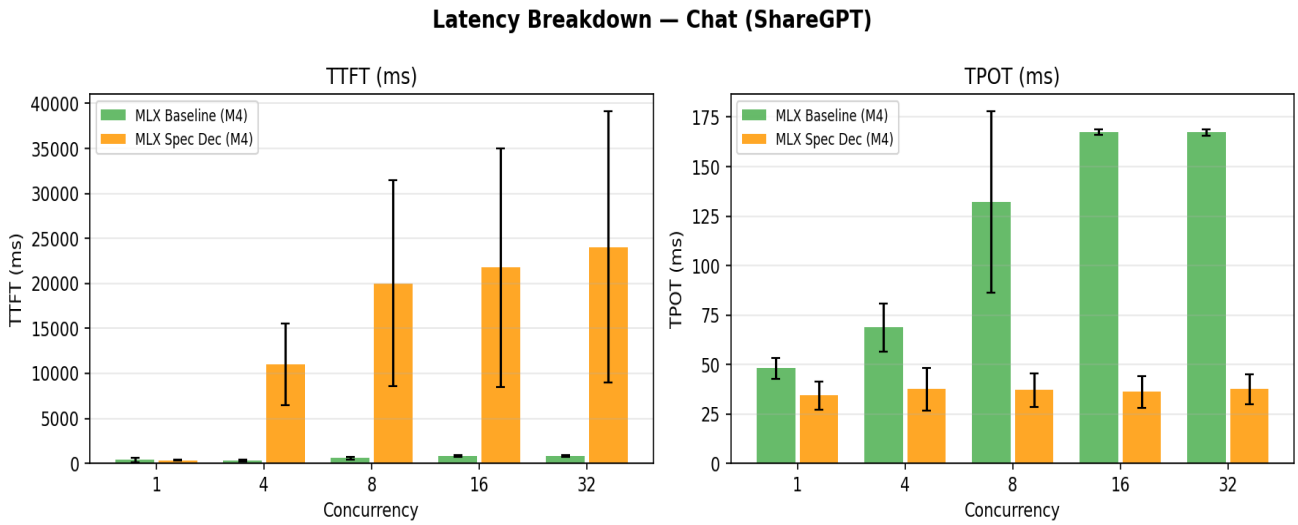

Figure 5: TTFT and TPOT breakdown for Chat (ShareGPT) at each concurrency level.

5.2 Code (HumanEval)

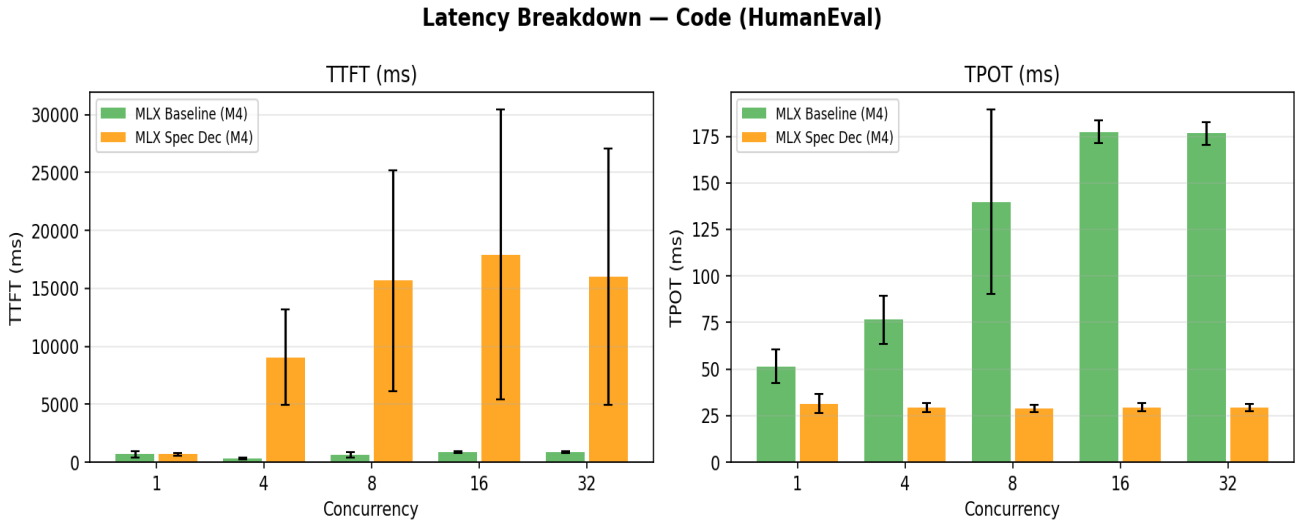

Figure 6: TTFT and TPOT breakdown for Code (HumanEval) at each concurrency level.

5.3 Summarization (CNN/DM)

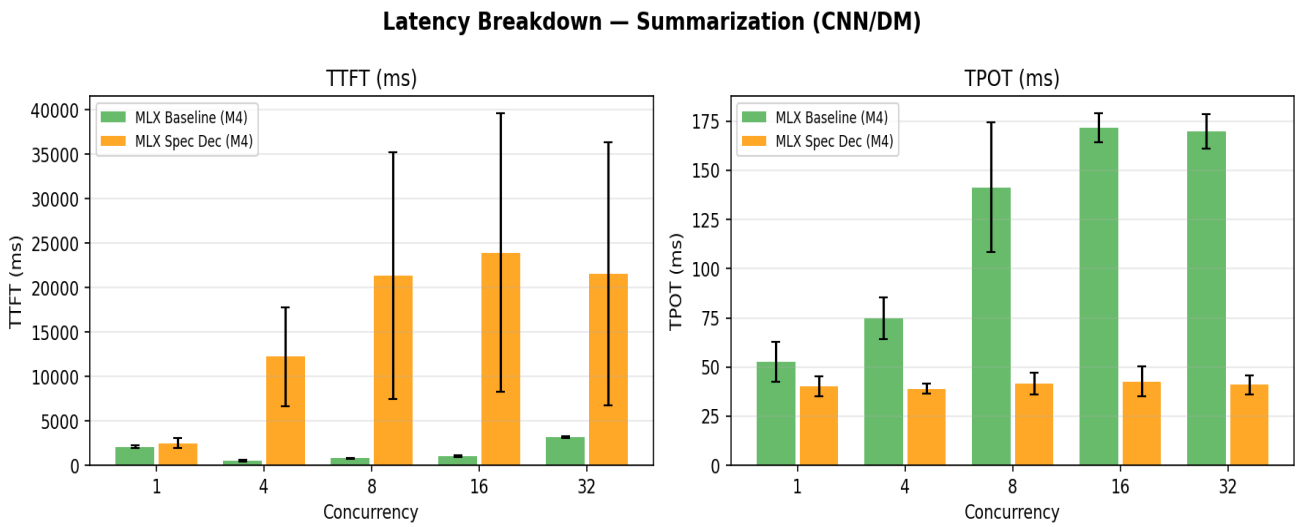

Figure 7: TTFT and TPOT breakdown for Summarization (CNN/DM) at each concurrency level.

6. Discussion & Implications

6.1 Why Speculative Decoding Helps at c=1

At concurrency = 1, the system is fully memory-bandwidth bound: generating the next token requires loading all 4.2 GB of model weights from unified memory on each step. Speculative decoding amortizes this cost by generating multiple draft tokens cheaply with the 1B model (~680 MB), then verifying a batch of tokens with the 8B model in a single forward pass. If the acceptance rate is reasonable, this effectively increases the number of tokens produced per weight-load cycle, improving throughput.

6.2 Why It Breaks at c≥4

MLX's server implements speculative decoding as a sequential per-request pipeline: draft all tokens → verify all tokens → emit. Unlike vLLM's PagedAttention, which can interleave multiple requests in a single batch, mlx-lm processes one speculative chain at a time. When 4 or more requests arrive simultaneously, each must wait for the entire preceding chain to complete before it receives its first token. At c=4, a request can wait through 3 full spec-dec pipelines before starting — explaining the 11-second TTFT observed.

6.3 Comparison: Mac vs GPU (EAGLE-3)

The GPU team running vLLM + EAGLE-3 on GCP L4/A100 hardware will observe a fundamentally different crossover profile for two reasons:

① **EAGLE-3 vs standard spec dec:** EAGLE-3 uses a feature-level draft head trained specifically on the target model's hidden states. Its acceptance rate is ~40–55% compared to ~15–25% for a generic 1B draft. This means more tokens accepted per verification step and higher speedup.

② **PagedAttention (vLLM):** vLLM's continuous batching and PagedAttention allow multiple requests to share GPU compute efficiently. Speculative decoding in vLLM runs draft and verify as part of the same batch schedule, so TTFT does not blow up at moderate concurrency the way it does in MLX.

> **Architectural Implication:** The Mac/MLX platform is best suited for *single-user interactive inference* (c=1) where speculative decoding provides meaningful benefit. For multi-user serving (c≥4), the baseline autoregressive server is strictly preferable on this hardware. GPU deployments with vLLM + EAGLE-3 remain beneficial up to much higher concurrency levels (expected c=8–16 crossover).

6.4 Limitations

Sample size: 10 prompts per combination limits statistical confidence. Standard deviations are high at some concurrency levels (especially c=4–8 for speculative, where TTFT variance is large due to queuing). A production benchmark would use 50–100 prompts.

Acceptance rate not measured: MLX does not expose the draft token acceptance rate through its server API. We cannot directly attribute speedup to acceptance rate on this hardware. The GPU team's vLLM Prometheus endpoint will provide this metric.

Single trial: Each combination was run once (trial=1). Thermal throttling on the M4 chip may affect later measurements in the sweep (particularly summarization, which runs last).

7. Conclusions

1. Speculative decoding works on M4 at c=1

The 1B draft + 8B target configuration delivers 1.4×–1.5× throughput improvement and 25–40% lower TPOT at single-user concurrency. This validates the core thesis that speculative decoding accelerates memory-bandwidth-bound inference.

2. The crossover point is sharp and early

The benefit disappears completely at concurrency=4. TTFT explodes to 10–24 seconds due to MLX's serial draft-verify scheduling. This is the defining limitation of speculative decoding on the Mac/MLX platform.

3. TPOT improvement persists at all concurrency levels

Even when TTFT makes the server impractical at high concurrency, individual tokens are generated faster by the speculative server. This is a genuine algorithmic benefit of speculative decoding, decoupled from scheduling concerns.

4. GPU + EAGLE-3 is expected to dominate at moderate concurrency

Based on the architectural analysis, vLLM's PagedAttention and EAGLE-3's higher acceptance rate will combine to push the crossover point significantly higher. The Mac data provides the lower-bound reference point for this comparison.

5. Platform choice should match workload

For single-user chat or IDE copilot workloads (c=1), Mac M4 + MLX speculative decoding is an excellent, cost-free (no cloud bill) alternative. For multi-user API serving, a GPU instance with vLLM is necessary.

Appendix A: Full Numeric Results

Complete mean metrics from all 300 request records (10 prompts × 2 systems × 3 tasks × 5 concurrency levels × 1 trial).

System	Task	Conc	TPS	TTFT ms	TPOT ms	Speedup
mlx_baseline	chat	1	20.0	399	48.0	
mlx_spec			28.0	335	34.4	**1.40×**
mlx_baseline	chat	4	14.7	313	68.6	
mlx_spec			9.3	10983	37.5	**0.63×**
mlx_baseline	chat	8	9.4	563	132.0	
mlx_spec			7.1	20007	37.0	**0.76×**
mlx_baseline	chat	16	5.8	828	167.4	
mlx_spec			6.8	21774	36.1	**1.18×**
mlx_baseline	chat	32	5.8	830	167.2	
mlx_spec			6.8	24018	37.6	**1.18×**
mlx_baseline	code	1	18.1	681	51.5	
mlx_spec			27.7	680	31.3	**1.53×**
mlx_baseline	code	4	13.2	312	76.5	
mlx_spec			11.9	9055	29.4	**0.91×**
mlx_baseline	code	8	8.9	642	139.8	
mlx_spec			9.5	15690	28.9	**1.06×**
mlx_baseline	code	16	5.5	846	177.4	
mlx_spec			9.5	17928	29.4	**1.73×**
mlx_baseline	code	32	5.5	840	176.6	
mlx_spec			10.0	16029	29.3	**1.82×**
mlx_baseline	summarization	1	14.0	2095	52.6	
mlx_spec			16.6	2474	40.1	**1.19×**
mlx_baseline	summarization	4	12.9	484	74.8	
mlx_spec			8.7	12238	39.0	**0.67×**
mlx_baseline	summarization	8	7.2	779	141.4	
mlx_spec			7.3	21335	41.5	**1.01×**
mlx_baseline	summarization	16	5.6	1051	171.6	
mlx_spec			6.2	23909	42.7	**1.11×**
mlx_baseline	summarization	32	5.0	3158	169.8	
mlx_spec			7.6	21565	41.0	**1.51×**

Appendix B: Environment Reproducibility

```
python3 -m mlx_lm.server --model mlx-community/Meta-Llama-3.1-8B-Instruct-4bit --port
8000
python3 -m mlx_lm.server --model mlx-community/Meta-Llama-3.1-8B-Instruct-4bit \
--draft-model mlx-community/Llama-3.2-1B-Instruct-4bit --port 8001

# Run sweeps
bash benchmark/run_sweep.sh mlx_baseline mac http://localhost:8000
bash benchmark/run_sweep.sh mlx_spec mac http://localhost:8001

# Generate report
python3 benchmark/plot_results.py
python3 generate_report.py
```